

SAPIENZA
UNIVERSITÀ DI ROMA

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data

Faculty of Information Engineering, Computer Science and Statistics
Applied Computer Science and Artificial Intelligence

Alessio Curcio
ID number 2120317

Supervisor
Danilo Avola

Co-supervisor
Daniele Bailo

Academic Year 2025/2026

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data
Bachelor's thesis. Sapienza University of Rome

© 2026 Alessio Curcio. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: alessio.curcio04@gmail.com

To my family, colleagues, and friends, by my side today or carved into my memories,
whose support is etched in the surface of every page,
shaping every step of this path.

Acknowledgments

First and foremost, I would like to express my gratitude to my academic supervisor, Prof. Danilo Avola, for his availability and for accepting to supervise my internship project. I am equally indebted to my internship supervisor, Dr. Daniele Bailo, for the trust he placed in me by welcoming me under his wing and for providing exceptional mentorship during this project.

I am profoundly grateful to Giada and Rossella for their tireless collaboration. Their constant readiness to share their knowledge and their unwavering moral support were truly the backbone of this work.

A very special thanks goes to Alessandro and all my friends from the "bunker". Thank you for being by my side through the many highs and lows of these past three years of my bachelor's degree. I also want to extend my most sincere appreciation to Gianmaria for his endless patience and invaluable help during our countless hours of study together.

Finally, my heartfelt gratitude goes to my family for their patience, emotional support, and for always believing in me.

Abstract

This thesis documents the design, implementation, and evaluation of **Seismic Viewer**, an advanced desktop application for four-dimensional (spatio-temporal) visualisation and analysis of seismological datasets. The system was developed during an internship focused on addressing a critical gap in modern seismology: the lack of accessible, high-performance tools capable of rendering massive hypocentral point clouds in real time while integrating spatial clustering and structural analysis.

Traditional seismic analysis relies predominantly on two-dimensional maps or static plots that fail to capture the intricate spatial and temporal relationships inherent in seismogenic sequences. Seismic Viewer addresses this limitation through a hybrid architecture that combines a Python-based computational backend, leveraging vectorised numerical computing, machine learning clustering algorithms, and robust geospatial processing, with a hardware-accelerated WebGL frontend rendered through an embedded Chromium engine.

The application supports four distinct visualisation modes: 2D geospatial mapping, 3D static hypocentral distribution, 4D time-animated sequences, and interactive cluster analysis with structural plane fitting. It integrates data from diverse sources, including local CSV files, the Italian Istituto Nazionale di Geofisica e Vulcanologia (INGV) GOSSIP API, GNSS uplift measurements, and seismic tomography datasets. Advanced algorithms, including auto-tuned DBSCAN and MiniBatchKMeans clustering, RANSAC-based fault plane identification, and a novel global label matching scheme for temporal stability, are employed to extract geological meaning from raw data.

This document presents the complete software engineering lifecycle: from domain analysis and architectural decisions through implementation details and performance optimisations, to the resolution of significant technical challenges encountered during development. The system has been validated using real-world datasets from the Campi Flegrei volcanic complex, demonstrating its capability to handle datasets exceeding 50,000 seismic events with sub-second interactive response times.

Contents

1 Introduction **1**
1.1 Motivation and Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives and Scope . . . 2
1.4 Internship Context . . . 2
1.5 Document Structure . . . 3

2 Background and Related Work **4**
2.1 Seismological Fundamentals . . . 4
2.1.1 Hypocentres and Epicentres . . . 4
2.1.2 Magnitude Scales . . . 4
2.1.3 Strike, Dip, and Dip Direction . . . 4
2.2 The Campi Flegrei Volcanic Complex . . . 5
2.3 Existing Seismic Visualisation Tools . . . 5
2.4 Machine Learning in Seismology . . . 6
2.4.1 Density-Based Clustering: DBSCAN . . . 6
2.4.2 Centroid-Based Clustering: K-Means . . . 7
2.5 Technology Choices and Justifications . . . 7
2.5.1 Why pywebview over Electron . . . 7
2.5.2 Why Plotly.js over D3.js and Three.js . . . 8
2.5.3 Why FastAPI over Flask . . . 8

3 System Architecture **9**
3.1 High-Level Architecture Overview . . . 9
3.2 Backend Architecture . . . 10
3.2.1 Application Entry Point & Bridge Engine . . . 10
3.2.2 Session Store & FastAPI Middleware . . . 10
3.3 Frontend Architecture . . . 11
3.3.1 Single-Page Application & UI/UX Design . . . 11
3.4 Data Flow Pipeline & Protocols . . . 12
3.4.1 Backend Data Processing . . . 12
3.4.2 Frontend Rendering and Communication . . . 13
3.4.3 Binary Data Transfer Protocol . . . 13
3.5 Threading and Concurrency Model . . . 13

4 Data Processing and Geospatial Engine **14**
4.1 CSV Ingestion and Schema Detection . . . 14
4.1.1 Adaptive Delimiter & Column Matching . . . 14
4.2 Data Sanitisation and Validation . . . 15
4.3 Online Data Import: The INGV GOSSIP API . . . 15
4.4 Geospatial Computations & Mesh Generation . . . 15
4.4.1 Inverse Mercator Projection . . . 15
4.4.2 UTM Zone 33N to WGS84 Conversion . . . 16
4.5 Optimised Map Mesh & Caching Architecture . . . 17
4.6 Visual Pre-computation & Optimization . . . 17
4.6.1 Asynchronous Binary Buffer Streaming . . . 17

5 Visualisation Modes **18**
5.1 2D Geospatial Map View . . . 18
5.1.1 Map Layers and Real-Time Filtering . . . 18
5.2 3D Static View . . . 20
5.2.1 Mesh Rendering & Aspect Ratio . . . 20
5.3 3D Animated View (4D Visualisation) . . . 22
5.4 3D Cluster Analysis View . . . 24
5.4.1 Cluster Visualisation & Interactive Tuning . . . 24
5.4.2 Sub-Clustering . . . 24
5.5 Additional Data Layers & View Caching . . . 27
5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations . . . 27
5.5.2 View Caching and Navigation . . . 28

6 Clustering and Structural Analysis Algorithms **29**
6.1 Coordinate Projection & Spatial Clustering Methods . . . 29
6.1.1 Metric Space Projection . . . 29
6.1.2 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning . . . 29
6.2 Automatic Parameter Tuning . . . 30
6.3 Structural Plane Fitting & Geometric Synthesis . . . 31
6.3.1 RANSAC Robust Plane Estimation . . . 31
6.3.2 SVD Refinement & Strike/Dip Mechanics . . . 31
6.4 Temporal Stability & Parallel Execution . . . 32

7 Development Process and Challenges **33**
7.1 Development Timeline & Iterative Methodology . . . 33
7.2 Key Technical Challenges and Solutions . . . 34
7.2.1 Rendering & UX Optimization: Camera Persistence, Binary Streaming, and Transitions . . . 34
7.2.2 Algorithmic Resilience & Memory Management: Temporal Flickering, API Isolation, and Caching . . . 34
7.3 Code Quality and Architecture . . . 35

8 Conclusions and Future Work **36**
8.1 Summary of Contributions 36
8.2 Validation & Performance Assessment 37
8.3 Future Research & Architectural Extensions 37
8.4 Concluding Remarks 38

A Supplementary Technical Details **39**
A.1 Seismic Data Structures 39
A.1.1 Input Catalog Format 39
A.1.2 Internal Application State 39
A.2 Algorithmic Pipeline 40
A.2.1 Spatial Clustering 40
A.2.2 Fault Plane Extraction 40

List of Figures

3.1 System architecture highlighting the three Python-Chromium communication channels. 9
3.2 The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b). . . 11
3.3 Linear data pipeline from input parsing to WebGL drawing. 12
3.4 Concurrency hierarchy linking UI, ASGI server, and analysis worker threads. 13

4.1 The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds. 16

5.1 The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c). 20
5.2 The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c). 22
5.3 The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c). 23
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1). 25
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2). 26

7.1 Development milestones highlighting the iterative implementation methodology. 33

List of Tables

2.1 Comparison of existing seismic visualisation tools against Seismic Viewer. 6
2.2 Summary of the selected technology stack and rejected alternatives. 7

3.1 REST API endpoints and WebSocket message protocol. 10
3.2 Binary buffer layout (Float32). 13

6.1 Comparison of spatial clustering approaches in Seismic Viewer. . . . 30

A.1 Expected schema for input seismic catalogs. 40
A.2 Internal data representation in the processing pipeline. 40

Chapter 1

Introduction

1.1 Motivation and Context

Seismology is among the most data-intensive fields within Earth sciences. Today's seismic monitoring networks continuously record thousands of events, generating extensive datasets that encode critical information about subsurface processes, such as tectonic stress accumulation and magmatic fluid migration. The capability to interpret these vast volumes of hypocentral data in near-real time goes beyond academic research. In fact, for regions characterized by elevated volcanic and seismic hazards, it represents a fundamental component of risk assessment and civil protection.

Traditionally, the analysis of earthquake catalogues has been conducted through two-dimensional maps, cross-sections, and time-series plots. While individually informative, these representations fail to capture the intricate four-dimensional nature of seismogenic processes. A cluster of earthquakes migrating along a plane at depth, for instance, may appear as a diffuse cloud in plan view yet reveal a coherent planar structure when examined in three dimensions and animated over time. This gap between the richness of the data and the poverty of conventional visualisation tools motivated the development of **Seismic Viewer**.

The project emerged from a direct need identified during an internship at the Istituto Nazionale di Geofisica e Vulcanologia (INGV). Researchers studying the Campi Flegrei caldera, one of the most densely monitored and potentially hazardous volcanic systems in the world, required a tool that could simultaneously display spatial distributions, temporal evolution, and structural relationships within seismic sequences, all within a fluid, interactive environment.

1.2 Problem Statement

Despite the availability of numerous Geographic Information System (GIS) platforms and seismological software packages, a significant technological gap persists. Existing tools generally fall into one of two categories:

(a) **General-purpose GIS platforms** (e.g., QGIS, ArcGIS): These offer powerful geospatial analysis capabilities but are not optimised for the fluid rendering of

dynamic 3D point clouds with seismic metadata. Their architectures prioritise cartographic accuracy over real-time interactivity, resulting in prohibitive latencies when manipulating datasets of tens of thousands of hypocentres.

(b) **Specialised seismological software** (e.g., Seisan, GMT, ObsPy): These provide robust analytical functionality but typically produce static outputs, such as publication-quality figures rather than interactive explorations. The learning curve is steep, requiring command-line proficiency, and the visualisation paradigm remains fundamentally two-dimensional.

Neither category adequately addresses the need for a tool that unifies *real-time 3D rendering*, *temporal animation*, *machine learning-based clustering*, and *structural geological analysis* within a single, accessible desktop application. Seismic Viewer was designed to fill this gap.

1.3 Objectives and Scope

The primary objectives of this project are:

1. **Interactive 4D Visualisation**: Develop a system capable of rendering seismic hypocentres in three spatial dimensions plus time, with smooth animations and real-time filtering.
2. **Automated Structural Analysis**: Integrate spatial clustering algorithms (DBSCAN, K-Means) and plane fitting (RANSAC + SVD) to automatically identify seismogenic structures.
3. **Multi-Source Data Integration**: Support ingestion from local CSV files, the INGV GOSSIP online API, seismic tomography datasets (Vp/Vs ratios), and GNSS ground deformation data.
4. **Performance at Scale**: Ensure sub-second interactive response times for large datasets, through binary data transfer, GPU-accelerated rendering, and efficient caching strategies.
5. **Accessible User Experience**: Provide a modern, intuitive interface that does not require programming expertise, enabling geophysicists to focus on interpretation rather than tool operation.

This report covers the entire development process of the application, from the initial design to the final testing and evaluation. The system was then tested using real earthquake data from the Campi Flegrei volcano to ensure it works well in practice.

1.4 Internship Context

This work was conducted as part of a university internship. The project was developed collaboratively with Giada Alfonsi, a PhD student in geology at Roma

Tre University, under the supervision of Daniele Bailo, a software engineer at the INGV.

An iterative development approach was adopted, progressively expanding the software's capabilities over the duration of the internship. The primary phases of the project were as follows:

- **Phase 1**: Initial prototype with basic 3D visualisation and animation capabilities.
- **Phase 2**: Major architectural refactoring; introduction of clustering (K-Means), GUI overhaul, online data import from INGV, and structural plane visualisation.
- **Phase 3**: Performance optimisation (binary streaming, cylinder-based clustering); project documentation.
- **Phase 4**: Sub-clustering feature; auto-tuning of DBSCAN parameters via K-distance analysis.
- **Phase 5**: Integration of tomography and GNSS uplift layers; final modular refactoring of the codebase.

1.5 Document Structure

The remainder of this thesis is organised as follows:

Chapter 2 reviews the seismological and technological background, surveys related work, and justifies the key technology choices.

Chapter 3 presents the system architecture, detailing the backend-frontend separation, communication protocols, and threading model.

Chapter 4 describes the data processing pipeline, including CSV ingestion, online import, geospatial computations, and performance optimisations.

Chapter 5 documents the four visualisation modes and the integration of auxiliary data layers (tomography, uplift, station positions).

Chapter 6 formalises the clustering and structural analysis algorithms, with mathematical derivations and implementation details.

Chapter 7 reflects on the development process, discusses key technical challenges, and presents the solutions adopted.

Chapter 8 summarises the contributions, acknowledges limitations, and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter outlines the theoretical and technical foundations of the project. It starts by introducing fundamental seismological concepts and presenting the Campi Flegrei volcanic complex, which serves as the primary case study for the application. Following a review of the current state of seismic visualisation tools, the chapter concludes by explaining and justifying the software tools selected for the development of Seismic Viewer.

2.1 Seismological Fundamentals

2.1.1 Hypocentres and Epicentres

An earthquake is characterised by the sudden release of accumulated elastic energy along a geological fault. The point at which rupture initiates within the Earth's crust is termed the **hypocentre** (or focus), defined by three coordinates: latitude (ϕ), longitude (λ), and depth (z). Its vertical projection onto the surface is the **epicentre**. While traditional seismological analyses often project data onto the surface plane, losing depth information, Seismic Viewer preserves the full three-dimensional hypocentral distribution.

2.1.2 Magnitude Scales

Earthquake size is quantified through magnitude scales. The datasets processed by Seismic Viewer primarily use the **duration magnitude** (M_D), which is the standard scale employed by the INGV for local events in the Campanian volcanic district. The system also supports generic magnitude fields (M_L, M_W) through its adaptive column-matching schema (see Section 4.1).

2.1.3 Strike, Dip, and Dip Direction

The geometry of a geological is conventionally described by three parameters:

- **Strike** (ϕ_s): The compass direction of the line formed by the intersection of the fault plane with a horizontal surface, measured clockwise from North.

- **Dip** (δ): The angle of maximum inclination of the fault plane relative to the horizontal, measured perpendicular to the strike direction.
- **Dip Direction**: The compass direction indicating the downward inclination of the fault plane. It is always perpendicular to the strike and is calculated as $(\phi_s + 90°) \bmod 360°$.

These parameters are central to the structural analysis module of Seismic Viewer, where they are computed automatically from clustered hypocentral distributions using the RANSAC and SVD algorithms described in Chapter 6.

2.2 The Campi Flegrei Volcanic Complex

The Campi Flegrei caldera, situated west of Naples in southern Italy, represents one of the most significant volcanic hazard areas in Europe. The caldera, approximately 13 km in diameter, hosts a population of over 500,000 people within its boundaries and lies adjacent to the metropolitan area of Naples.

Since 2005, the Campi Flegrei have exhibited a sustained episode of ground uplift (*bradyseism*), accompanied by increasing seismicity. The INGV Osservatorio Vesuviano continuously monitors the area through a dense network of seismic stations, GNSS receivers, and geochemical sensors. The resulting datasets, comprising tens of thousands of located earthquakes, daily GNSS displacement time series, and periodic tomographic inversions, constitute the primary validation domain for Seismic Viewer.

Understanding the dynamics at Campi Flegrei, such as magmatic fluid movements, hydrothermal activity, and fault reactivation, requires software that can combine different data types in a single spatial and temporal environment. This necessity was the main objective behind the multi-layer architecture of Seismic Viewer (see Section 5.5). Furthermore, recent studies [3] have demonstrated that the spatial distribution of seismicity in this caldera can be effectively modelled and represented through planar structures, highlighting the birth and growth of volcanotectonic faults. This finding strongly motivates and justifies the integration of specific fault plane fitting algorithms (such as RANSAC and SVD) into the core analysis module of the application.

2.3 Existing Seismic Visualisation Tools

QGIS and ArcGIS. While excellent for 2D cartography, their architecture is tailored for vector and raster layers rather than dynamic 3D point clouds. Although plugins like Qgis2threejs add 3D capabilities, performance drops when handling thousands of features. Furthermore, creating time-based animations or clustering data requires complex workarounds.

GMT and PyGMT. These tools are highly regarded for producing publication-quality maps and cross-sections. However, their script-based, batch-processing workflow lacks interactivity. Regenerating outputs after every parameter tweak is simply too slow for the rapid, iterative exploration needed to analyse seismic sequences.

Seisan. This package is widely used for processing seismic data. However, it only offers basic 2D visualisations, and its older codebase makes it difficult to modernise or extend.

ParaView. While ParaView provides excellent 3D graphics and animations, it is a general-purpose tool. It lacks specific features for seismology, such as automatic clustering, fault plane analysis, or direct integration with seismic databases like the INGV GOSSIP API.

Table 2.1 summarises the capabilities of established tools and highlights the gaps that Seismic Viewer addresses.

Table 2.1. Comparison of existing seismic visualisation tools against Seismic Viewer.

Tool	3D View	4D Anim.	Cluster	Real-time	GUI	Primary Limitations
QGIS	Limited	×	Plugin	×	✓	Not optimised for dense 3D point clouds
GMT / PyGMT	✓	×	×	×	×	Static output only, command-line workflow
Seisan	✓	×	×	×	Partial	Legacy codebase, steep learning curve
ObsPy	✓	×	External	×	×	Library framework, not an interactive GUI
GPlates	✓	✓	×	Partial	✓	Focused on global plate tectonics, not local swarms
ParaView	✓	✓	×	Partial	✓	General-purpose, lacks seismological workflows
Seismic Viewer	✓	✓	✓	✓	✓	Domain-specific, standalone local deployment

The analysis above reveals that no existing tool integrates all five capabilities, interactive 3D rendering, temporal animation, automated clustering, real-time parameter adjustment, and seismic-specific analysis, within a single, accessible application. Seismic Viewer was designed precisely to fill this gap.

2.4 Machine Learning in Seismology

The application of machine learning to seismological data has grown substantially in recent years, largely supported by robust frameworks such as scikit-learn. Two families of algorithms are particularly relevant to the spatial analysis of hypocentral distributions:

2.4.1 Density-Based Clustering: DBSCAN

The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm identifies clusters as regions of high point density separated by regions of low density. Unlike partitioning methods, DBSCAN does not require the number of clusters to be specified *a priori* and can discover clusters of arbitrary shape, a critical advantage when analysing seismicity aligned along non-planar fault surfaces.

The algorithm is governed by two parameters: ε (the neighbourhood radius) and `MinPts` (the minimum number of points required to form a dense region). Points outside these clusters are flagged as **noise**, which naturally filters out scattered background seismicity to isolate the main seismogenic structures.

2.4.2 Centroid-Based Clustering: K-Means

The K-Means algorithm partitions a set of N observations into K clusters, minimising the within-cluster sum of squared distances to each cluster's centroid. Seismic Viewer employs the **MiniBatchKMeans** variant, which processes random subsets (mini-batches) of the data during each optimisation step. This modification reduces the computational complexity from $O(NKT)$ to near-linear time, enabling real-time reclustering as the user adjusts parameters interactively.

K-Means is preferred when the analyst has prior knowledge of the expected number of seismogenic structures and desires spherical cluster geometries.

2.5 Technology Choices and Justifications

To choose the technology stack, various alternatives were systematically compared, focusing on a balance between high performance, rapid development, and future maintainability. Table 2.2 summarises the key decisions.

Table 2.2. Summary of the selected technology stack and rejected alternatives.

Component	Chosen	Rejected	Justification
Language	Python 3.10+	Java, C++	Rich ecosystem for data science, rapid prototyping
Desktop Framework	pywebview	Electron, Qt	Lightweight memory footprint, native file dialogs, direct Python to JS bridge
HTTP Server	FastAPI	Flask, Django	Native async support, WebSockets, automatic API documentation, high throughput
Rendering	Plotly.js	D3.js, Three.js, CesiumJS	High-level 3D API, native WebGL acceleration, built-in interactivity
Clustering	scikit-learn	Custom, R	Reliable and highly optimized implementations, includes MiniBatchKMeans and DBSCAN
Maps	Smopy (OSM)	Mapbox, Google Maps	Free and open-source, requires no API key, supports offline caching

2.5.1 Why pywebview over Electron

To provide a smooth desktop experience without consuming excessive system resources, **pywebview** was chosen over Electron. Electron is a popular framework, but it bundles a full web browser and a Node.js runtime into the application. This

makes the software very heavy and demanding on memory, which is not ideal for a tool that already requires significant resources for data processing.

Conversely, pywebview offers a much lighter solution by relying on the web engine already installed on the user's operating system. This ensures that the application remains fast and uses memory efficiently.

Furthermore, pywebview includes a direct communication channel between the user interface and the Python backend. This feature simplifies the architecture, as standard desktop operations (such as opening files) can be handled directly, leaving the main server free to focus entirely on transferring large amounts of seismic data.

2.5.2 Why Plotly.js over D3.js and Three.js

Rendering 3D point clouds requires a library capable of handling large datasets efficiently. D3.js is excellent for standard two-dimensional charts, but it lacks built-in tools for 3D environments, making it too complex for this specific use case. On the other hand, Three.js is a powerful 3D engine, but it is too generic, implementing scientific features like coordinate axes and color scales would require writing a large amount of custom code.

Plotly.js proved to be the ideal compromise. It is specifically designed for scientific visualization and includes ready-to-use 3D components. Since it relies on hardware acceleration (WebGL), it guarantees smooth performance even with thousands of points. Additionally, it offers native interactive features such as zooming and rotating, allowing the development process to focus on data analysis rather than graphics programming.

2.5.3 Why FastAPI over Flask

The backend of Seismic Viewer needs to handle multiple tasks simultaneously, such as serving data, maintaining active connections for real-time animations, and running heavy clustering algorithms. Flask is a standard choice for web applications, but its traditional architecture struggles with concurrent operations. When executing a long calculation, Flask tends to block the entire application, causing delays in the user interface.

To avoid these bottlenecks, **FastAPI** was adopted. It is built from the ground up to support asynchronous operations, meaning it can handle multiple requests at the same time without freezing. This allows the system to easily offload heavy computations to the background while smoothly transmitting the results to the user interface through continuous data streams.

Chapter 3

System Architecture

This chapter presents the software architecture of Seismic Viewer, detailing the separation between computation and presentation, the communication protocols linking them, and the concurrency model ensuring interactive responsiveness.

3.1 High-Level Architecture Overview

Seismic Viewer follows a **dual-process hybrid architecture** combining a Python scientific processing backend with an embedded Chromium-based frontend. The layers communicate via three distinct channels tailored to specific operational requirements.

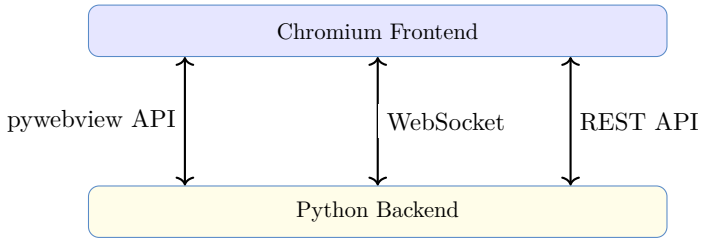


Figure 3.1. System architecture highlighting the three Python-Chromium communication channels.

- **pywebview Bridge** (`js_api`): This channel handles synchronous method calls between Python and JavaScript. It is primarily dedicated to low-latency graphical user interface (GUI) interactions, such as opening native file dialogs, managing application navigation, and handling configuration settings.
- **FastAPI REST API**: Designed for high-throughput data transfer, this channel exposes HTTP endpoints that deliver both session metadata (via `/api/data/{id}`) and fully pre-rendered HTML visualisations (via `/viz/{id}`). To ensure optimal performance with large datasets, it employs GZip compression and transmits heavy numerical arrays as raw Float32 binary buffers.
- **WebSocket**: This bidirectional communication protocol is essential for continuous, asynchronous data exchange. It is specifically used to push real-time

progress updates to the user interface during long-running tasks, as well as to handle interactive features like dynamic spatial analysis and the calculation of structural planes.

Table 3.1. REST API endpoints and WebSocket message protocol.

Protocol / Endpoint	Direction / Path	Description / Message Types
REST (`GET`)	`/api/data/{session_id}`	Retrieve session data as GZip-compressed JSON
REST (`GET`)	`/api/data/{session_id}/bin`	Retrieve numerical data as Float32 binary buffer
REST (`GET`)	`/viz/{viz_id}`	Serve pre-rendered HTML visualisation page
WebSocket	Client → Server	`ping`, `get_cluster_update`, `get_structural_planes`, `get_animated_clustering`
WebSocket	Server → Client	`pong`, `rpc_response`, `progress` (broadcast messages)

3.2 Backend Architecture

3.2.1 Application Entry Point & Bridge Engine

Once the application is running, the `Api` module acts as the central coordinator. Whenever the user interacts with the interface, such as selecting a file (`select_csv()`), loading a 3D view (`load_view()`), or exporting results (`export_clusters_csv()`), the frontend sends a request directly to this coordinator.

Many of these requests trigger computationally intensive mathematical operations, including advanced spatial analysis of seismic events. If the software were to perform these calculations on its main execution thread, the entire graphical interface would become unresponsive until the tasks were completed. To prevent this behavior, all heavy processing is delegated to background workers using the `threading.Thread` module.

Since the calculations occur in the background, the application requires a mechanism to inform the user about the ongoing progress. The coordinator handles this requirement by continuously transmitting real-time updates back to the interface. This ensures the user is constantly provided with a responsive progress bar or status message, resulting in a fluid and stable user experience. These updates are broadcast through the WebSocket protocol.

3.2.2 Session Store & FastAPI Middleware

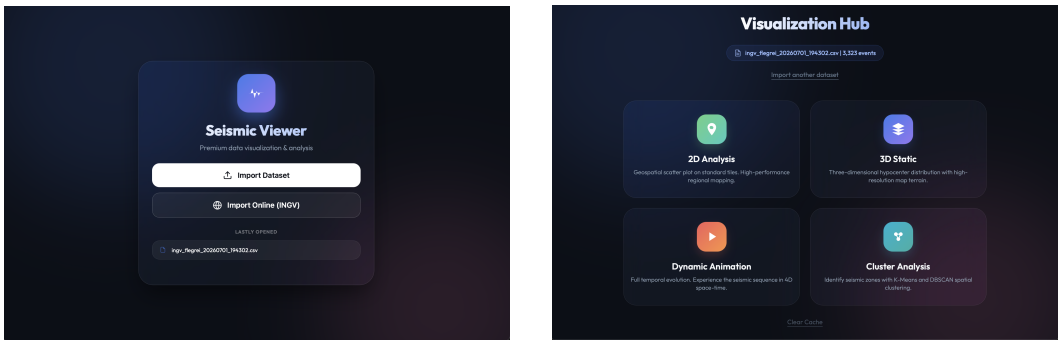
The system architecture requires a dedicated memory management layer to preserve state between user interactions and to serve data efficiently to the graphical frontend. This functionality is centralized within an internal configuration module (`core/config.py`), which orchestrates two separate in-memory caches: `_data_store` and `_html_store`.

The `_data_store` structure retains the raw numerical arrays and metadata associated with each active analytical session. To guarantee that the application maintains a stable memory profile even during extensive analytical workflows, this cache is strictly limited to 30 concurrent sessions. In parallel, the `_html_store` caches up to 15 pre-rendered graphical views, allowing the frontend to load complex tridimensional scenes instantaneously. Both structures implement a First-In, First-Out (FIFO) eviction policy. This strategy effectively bounds the overall Random Access Memory utilization without requiring manual intervention from the user.

Positioned between this caching layer and the client interface, the FastAPI server deploys specific middleware to optimize network throughput. The most critical optimization is the automatic application of GZip compression for any outgoing data payload larger than 500 bytes. Compressing the data prior to transmission significantly accelerates the rendering process, particularly when transferring datasets containing tens of thousands of seismic events.

3.3 Frontend Architecture

3.3.1 Single-Page Application & UI/UX Design



(a) File Picker view. **(b)** Visualisation Hub with mode selection.

Figure 3.2. The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b).

As illustrated in Figure 3.2, the primary workflow begins at the landing screen (a). Here, the system exposes multiple data ingestion pathways, allowing researchers to import local csv files, query the INGV remote web services directly, or restore a previously interrupted session through local historical records.

Once the imported dataset is validated, the software transitions to the visualization hub (b). This primary analytical workspace features specific menus to alternate between the available visualization and analysis algorithms. Furthermore, it incorporates interactive toggles to overlay supplementary spatial context onto the 3D map, facilitating the visual integration of seismic tomography models, GNSS deformation grids, and local monitoring stations.

The 3D rendering pipeline delegates all plotting operations to the WebGL graphics API via the Plotly framework. To ensure a fluid user experience during dynamic operations, the rendering layer is designed to preserve the specific spatial orientation

and zoom level chosen by the user. This prevents the virtual camera from resetting its spatial position when new data arrives, allowing the system to continuously update and re-process the underlying data in real time without interrupting the user's exploration (the technical implementation of this persistence is detailed in Section 7.2.1).

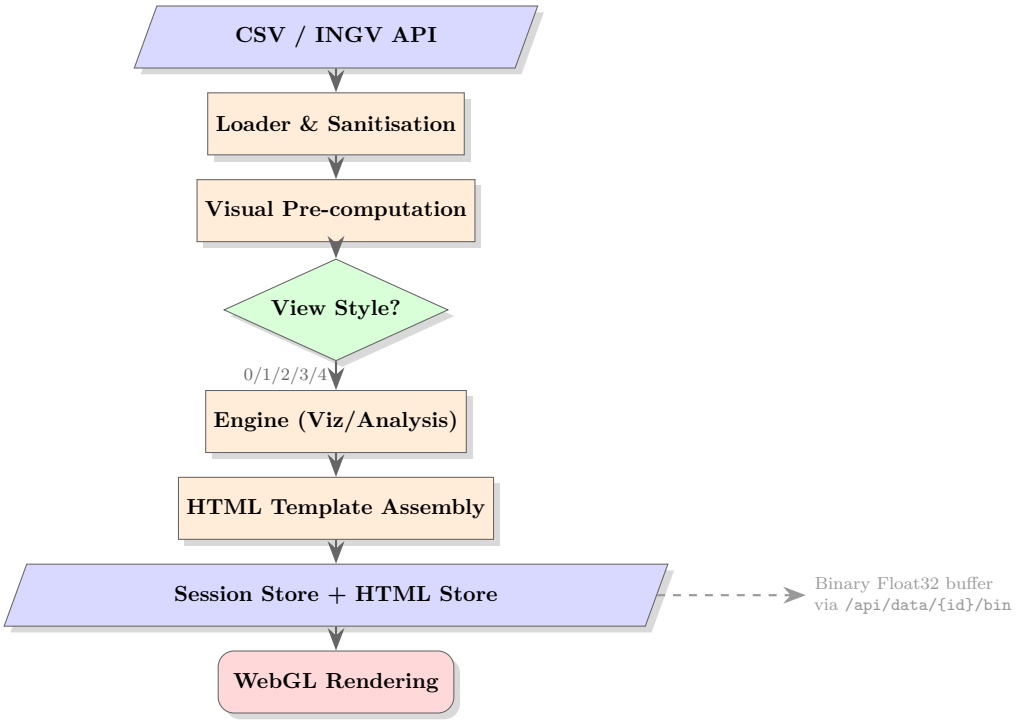


Figure 3.3. Linear data pipeline from input parsing to WebGL drawing.

3.4 Data Flow Pipeline & Protocols

In order to manage the entire data lifecycle within the application, Seismic Viewer divides its data flow operations clearly between the backend services and the frontend client. This separation is fundamental to maintain performance and ensure responsiveness.

3.4.1 Backend Data Processing

The backend architecture is responsible for all heavy lifting regarding data ingestion, validation, and complex mathematical analysis. As outlined in Figure 3.3, the pipeline begins by importing the CSV dataset or fetching remote data from the INGV API. This data undergoes an initial sanitization and loading phase. Following this, visual pre-computation prepares the structural elements before the engine applies the required analytical algorithm chosen by the researcher. Finally, the results are assembled into an HTML template and stored in the internal `_data_store` and `_html_store` caches, ready to be served.

3.4.2 Frontend Rendering and Communication

Once the backend processing is complete, the frontend retrieves the data through the defined REST API and WebSocket endpoints (Table 3.1). The frontend dynamically renders the 3D scene using WebGL via the Plotly.js library. Continuous status updates, real-time recalculations, and filtering commands are exchanged over the WebSocket channel, ensuring the interface remains fully interactive without needing page reloads.

3.4.3 Binary Data Transfer Protocol

To eliminate JSON parsing overhead during large catalog transfers, the backend packs numerical coordinates into a contiguous Float32 buffer (`/api/data/{id}/bin`) structured as: header `[event_count, cluster_count]` followed sequentially by arrays for longitude, latitude, magnitude, depth, `dates_num`, and cluster labels (C floats).

Table 3.2. Binary buffer layout (Float32).

Offset	Content	Offset	Content
0 (2 floats)	Header: `[N, C]`	$2+3N$ (N floats)	Depth array
2 (N floats)	Longitude array	$2+4N$ (N floats)	Dates_num array
$2+N$ (N floats)	Latitude array	$2+5N$ (C floats)	Cluster labels (if present)
$2+2N$ (N floats)	Magnitude array		

On the frontend, decoding via JavaScript's `Float32Array` directly over an `ArrayBuffer` completes in under 5 ms for 50,000 events (reducing transfer volume by approximately 90% and bypassing over 800 ms of JSON parsing and garbage collection delays).

3.5 Threading and Concurrency Model

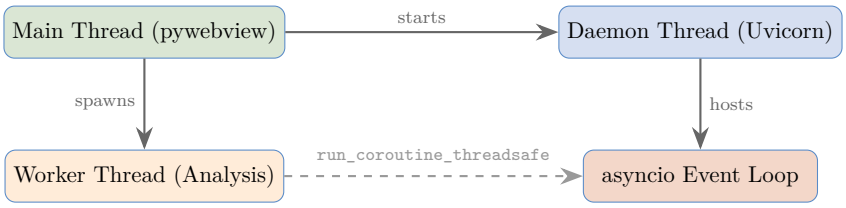


Figure 3.4. Concurrency hierarchy linking UI, ASGI server, and analysis worker threads.

The concurrency architecture separates responsibilities across the **Main Thread** (hosting pywebview and Chromium), the **Server Thread** (running the Uvicorn/asyncio event loop), and on-demand **Worker Threads** handling view generation and clustering updates. For intensive multi-frame calculations (animated clustering and RANSAC+SVD plane tracking), the engine escalates to a `ProcessPoolExecutor` with up to 8 worker processes utilizing shared memory initializers to avoid array serialization.

Chapter 4

Data Processing and Geospatial Engine

This chapter details the processing pipeline that transforms raw seismic observations (whether sourced from local catalogs or remote web services) into optimized, GPU-ready structures for interactive 3D and 4D visualization.

4.1 CSV Ingestion and Schema Detection

Seismic catalogues lack a universal file standard. Institutions employ varying column names, delimiters, date formats, and coordinate systems. The `core/loader.py` module addresses this heterogeneity via an adaptive ingestion pipeline:

4.1.1 Adaptive Delimiter & Column Matching

The loader scans the header line to discern between semicolon (`;`) and comma (`,`) delimiters, accommodating European standard dialects (INGV) alongside international databases (USGS, ISC). To avoid mandatory manual preprocessing, `core/config.py` defines a flexible alias dictionary:

```
CSV_SCHEMA = {
    'time':  ['time', 'date', 'datetime', 'utc', 'time_utc', '
    origintime'],
    'lat':   ['lat', 'latitude'],
    'lon':   ['lon', 'longitude'],
    'depth': ['depth', 'depth_km'],
    'mag':   ['mag', 'magnitude', 'md', 'ml', 'mw']
}
```

Listing 4.1. Flexible column schema definition in `core/config.py`.

The ingestion pipeline identifies data columns by reading their headers. It ignores text case and removes any extra spaces. When a file contains unlabelled columns, the loader assumes a standard INGV column order based on their positions. Date processing is equally robust. The engine tests multiple known formats, covering both standard ISO timestamps and common European layouts.

4.2 Data Sanitisation and Validation

Following ingestion, records undergo a four-stage sanitisation and validation sequence:

1. **Spatial requirement check**: Rows missing latitude, longitude, or timestamps are discarded immediately.

2. **Missing metadata flagging**: Catalogues occasionally omit depth or magnitude. Rather than dropping these records (which removes critical spatial density) or imputing synthetic values (which biases clustering), missing fields are flagged via boolean indicators (`depth_is_missing`, `mag_is_missing`) and assigned sentinel defaults (0.0 km depth and magnitude 0.1).

3. **Geographic and depth bounds check**: The system excludes any event with mathematically invalid geographic coordinates (i.e., latitude outside the $\pm 90^\circ$ range or longitude outside the $\pm 180^\circ$ range). Furthermore, it discards events with a depth falling outside the realistic geological range of -10 km to 800 km, where negative values are specifically permitted to accommodate earthquakes occurring above sea level within mountains or volcanic edifices.

4. **Chronological sorting & future filtering**: Events with future timestamps are discarded, and the remaining catalog is sorted chronologically to optimize slicing.

4.3 Online Data Import: The INGV GOSSIP API

The `core/online_importer.py` module interfaces directly with the INGV GOSSIP web service, providing high-resolution volcanic monitoring catalogues for Campi Flegrei, Vesuvius, and Ischia:

The importer queries the official INGV GOSSIP web portal for each selected year (using the endpoint `https://terremoti.ov.ingv.it/gossip/<network>/<year>/events.json`). It downloads these annual JSON packets one by one to avoid network failures. As shown in Figure 4.1, users can define the network target, the time span, the magnitude limits (M_{min} to M_{max}), and the desired INGV revision quality (`automatico`, `rivisto`, or `bollettino`). Moreover, they can apply additional filters to require confirmed coordinates (`located only`) or to drop records with incomplete information (`exclude missing`). Finally, the system converts these filtered records into a standard temporary CSV format and forwards them directly to the main ingestion pipeline.

4.4 Geospatial Computations & Mesh Generation

4.4.1 Inverse Mercator Projection

The 3D application visualizes seismic activity over a base map retrieved from OpenStreetMap. To accurately draw the earthquakes over this map, the software must convert the image pixel positions into standard geographic coordinates. The system solves this problem using an **inverse Mercator projection** (implemented

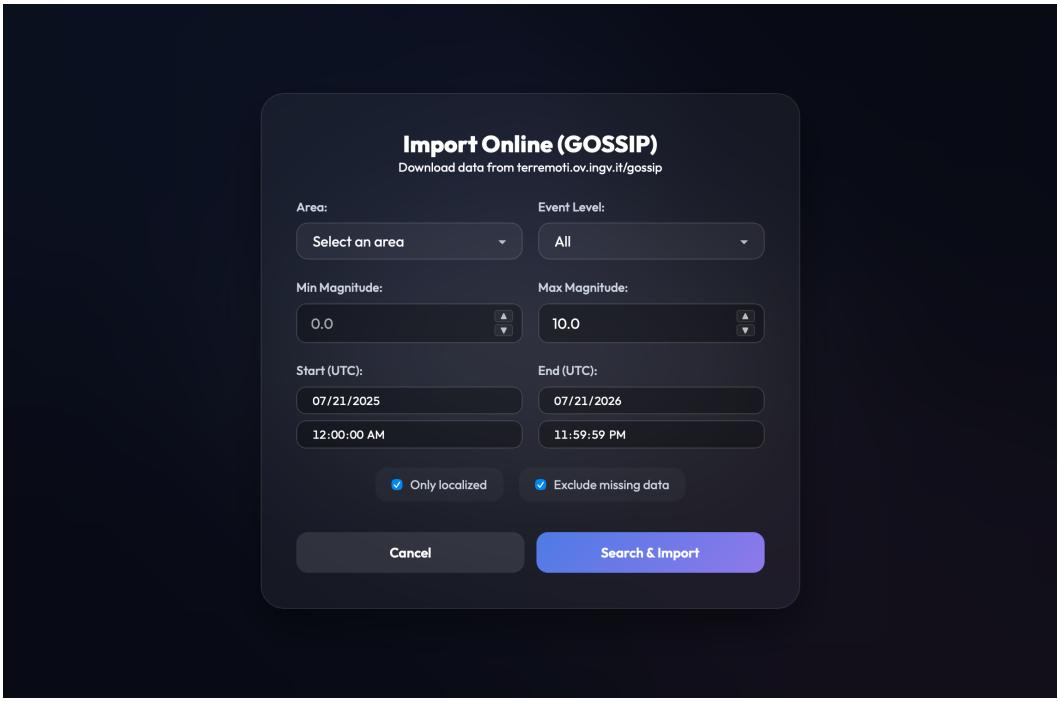


Figure 4.1. The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds.

in `core/geo_utils.py`). This mathematical operation translates pixel data into latitude and longitude, using NumPy arrays to guarantee fast execution:

$$\lambda = \frac{x \cdot s}{t \cdot n} \cdot 360^\circ - 180^\circ, \quad \phi = \arctan\left(\sinh\left(\pi\left(1 - \frac{2 \cdot y \cdot s}{t \cdot n}\right)\right)\right) \cdot \frac{180^\circ}{\pi} \tag{4.1}$$

In these equations, λ and ϕ represent the resulting longitude and latitude in degrees. The values x and y correspond to the horizontal and vertical pixel coordinates on the screen. The variable s is the current scaling factor of the map, while t represents the fixed dimension of a single map tile (256 pixels). Lastly, n is the total number of tiles across the map at the current zoom level z, calculated as $n = 2^z$.

4.4.2 UTM Zone 33N to WGS84 Conversion

Some advanced datasets, such as volcanic tomography grids, are originally provided in UTM Zone 33N coordinates instead of standard latitude and longitude. To ensure these layers align correctly with the rest of the 3D map, the `core/layer_utils.py` module converts them using a Transverse Mercator inverse projection on the WGS84 ellipsoid:

$$\phi = \phi_1 - \frac{N_1 \tan \phi_1}{R_1}\left(\frac{D^2}{2} - \frac{(5 + 3T_1 + 10C_1 - 4C_1^2 - 9e'^2)D^4}{24}\right) \tag{4.2}$$

Here, ϕ is the final converted latitude. The formula relies on several variables derived from the WGS84 ellipsoid model. Specifically, ϕ_1 is the footprint latitude, N_1 is

the prime vertical radius of curvature, and R_1 is the meridional radius of curvature at ϕ_1. The parameter D represents the scaled distance from the central meridian, calculated as $D = (E - 500{,}000)/(N_1 k_0)$, where E is the UTM Easting coordinate and k_0 is the central scale factor. Finally, T_1, C_1, and e'^2 are standard geometric constants related to the Earth's eccentricity.

4.5 Optimised Map Mesh & Caching Architecture

Terrain generation operates via a five-stage sequence: (1) bounding box squaring for uniform axis resolution, (2) deterministic cache lookup via MD5 hashing of rounded geographic bounds, (3) OpenStreetMap tile downloading via Smopy on cache misses, (4) palette-indexed downsampling to 1024×1024 px (256 colors), and (5) grid transformation via Equation (4.1). Meshes are cached both in RAM (`@lru_cache(maxsize=4)`) and disk (`.pkl`), guaranteeing sub-millisecond retrieval on subsequent loads.

4.6 Visual Pre-computation & Optimization

To ensure the 3D map remains fluid and responsive even with thousands of earthquakes, the software performs several heavy calculations in advance (during the data loading phase) rather than while the user is interacting with the map. This is handled by the `precompute_visual_data()` function:

- **Tooltip Pre-rendering**: Instead of forcing the user's browser to piece together the text for the pop-ups (e.g., "Magnitude: 3.5, Depth: 10 km") every time the mouse hovers over an earthquake, Python prepares all these text strings in advance in one quick batch operation.
- **Normalised Time Metrics**: Real calendar dates are mathematically converted into days since the first earthquake. This makes it much easier and faster for the rendering engine to assign the correct colors to earthquakes based on their chronological order.

4.6.1 Asynchronous Binary Buffer Streaming

Traditionally, embedding massive amounts of earthquake data directly into a static HTML file makes the file huge and causes the web browser to freeze while opening it. To solve this, the Seismic Viewer employs a hybrid approach.

When the user requests a 3D view, the Python backend initially generates a very lightweight HTML page. This page contains only the basic 3D environment, the terrain maps, and the user interface menus, allowing the browser to load and display the scene faster.

Immediately after the base map appears, the browser automatically makes a background request (`/api/data/{id}/bin`) to fetch the actual earthquake points. These points are sent in a highly compressed Float32 binary buffer format. Because the data is compressed and loaded in the background, thousands of earthquakes populate the map in milliseconds without freezing the user interface.

Chapter 5

Visualisation Modes

Seismic Viewer provides four primary visualisation modes, each addressing a distinct analytical need. This chapter documents their implementation, rendering strategies, and integration with auxiliary geophysical data layers.

5.1 2D Geospatial Map View

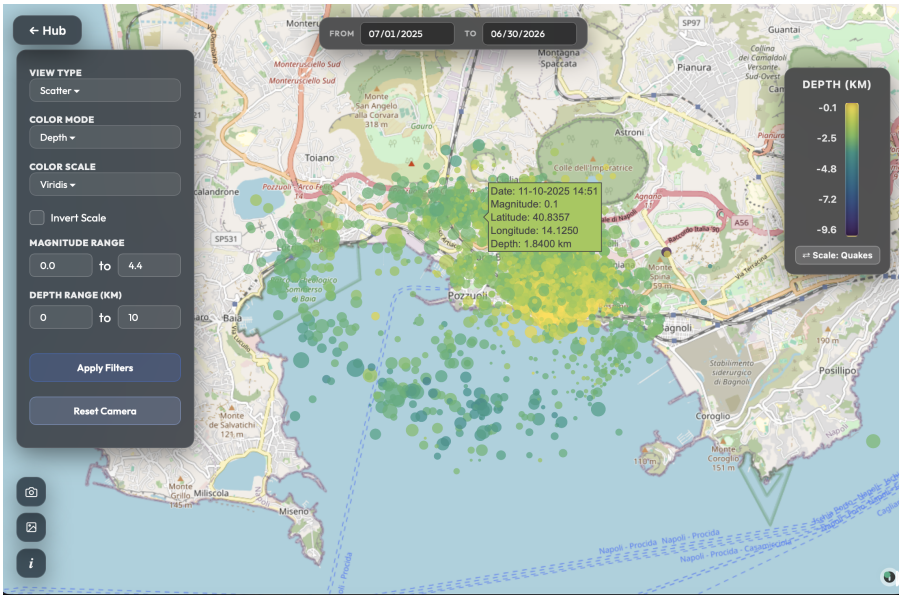
The 2D view provides a plan-view representation of earthquake epicentres overlaid on OpenStreetMap tiles, implemented through Plotly's `scatter_mapbox` trace type.

5.1.1 Map Layers and Real-Time Filtering

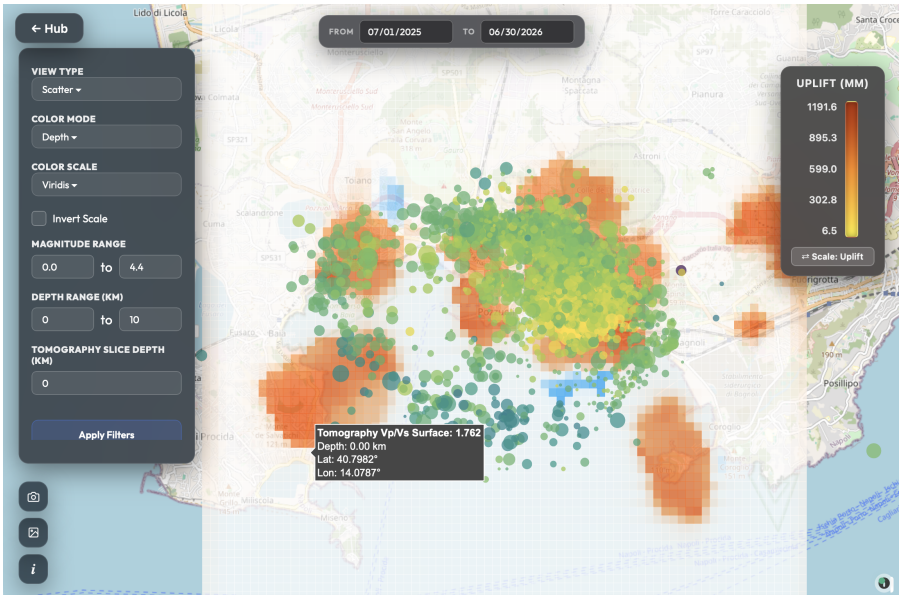
The 2D map visualizes the seismic data using two distinct graphical layers:

- **Scatter Markers**: Each earthquake is represented by a circular marker. The visual properties of these circles can be mapped to different physical parameters depending on the analytical needs. By default, the size of the circle indicates the magnitude of the event (with a minimum size threshold to ensure extremely weak micro-earthquakes remain visible), and the color represents the hypocentral depth. However, these mappings are highly flexible, allowing users to visualize other variables, such as chronological progression. Users can interact with these markers by hovering over them to reveal detailed statistics.
- **Density Heatmap**: A spatial density layer that visually aggregates tightly clustered earthquakes. By rendering a continuous color gradient over regions of high seismicity, this optional overlay allows researchers to intuitively identify active swarm locations and broader structural trends that might be difficult to perceive when analyzing individual scatter points alone.

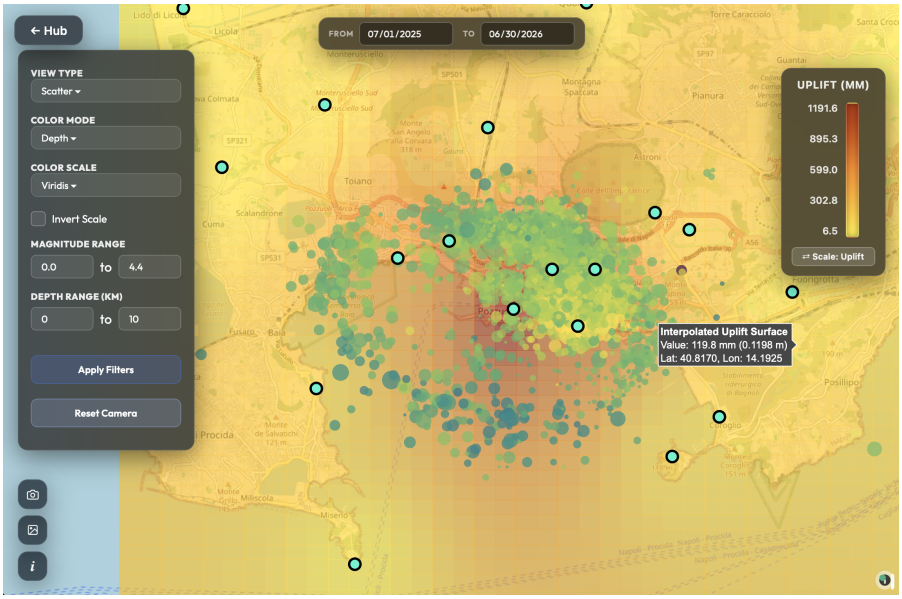
To explore the dataset, users can apply filters based on magnitude, depth, and date ranges using dedicated input fields located on the left-hand panel. All filtering operations are processed directly within the user's web browser. This approach avoids sending new requests to the main server, guaranteeing that the map updates instantly whenever a parameter is changed.



(a) The 2D base map showing earthquake locations. On the left panel, input fields allow the user to filter the data by magnitude, depth, and date.



(b) An overlay showing a tomography slice of the crust. The highlighted areas indicate high V_p/V_s velocity anomalies.



(c) An overlay displaying vertical ground movement (uplift) using a grid map, along with the locations of permanent monitoring stations across Campi Flegrei.

Figure 5.1. The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c).

As shown in Figure 5.1, the 2D workspace allows researchers to combine earthquake locations with other geological datasets. For example, Figure 5.1b displays a tomography slice where V_p/V_s velocity anomalies indicate the possible presence of hydrothermal fluids or magma. Furthermore, Figure 5.1c shows the cumulative vertical ground uplift alongside the monitoring stations. This combined view helps geophysicists understand if earthquake swarms are occurring in the same areas where the caldera is inflating the most.

5.2 3D Static View

The 3D static view renders hypocentres as a subsurface point cloud within a three-dimensional geographic volume beneath an OpenStreetMap terrain surface.

5.2.1 Mesh Rendering & Aspect Ratio

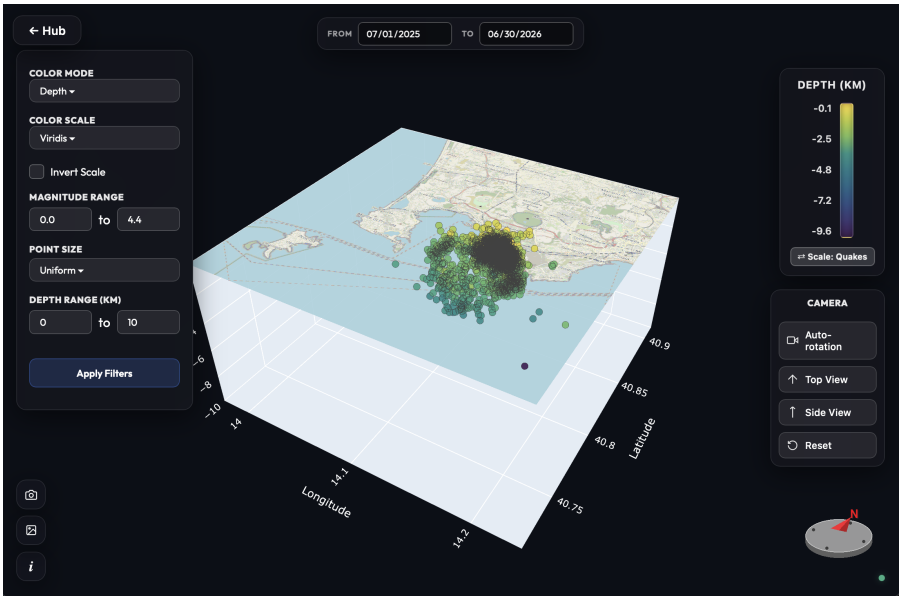
The reference terrain is rendered as a flat, unshaded map layer for clarity. Earthquakes are represented beneath the surface as a 3D point cloud, color-coded by variables like depth or time.

To prevent geographic distortion caused by Earth's sphericity, the system scales horizontal proportions based on latitude:

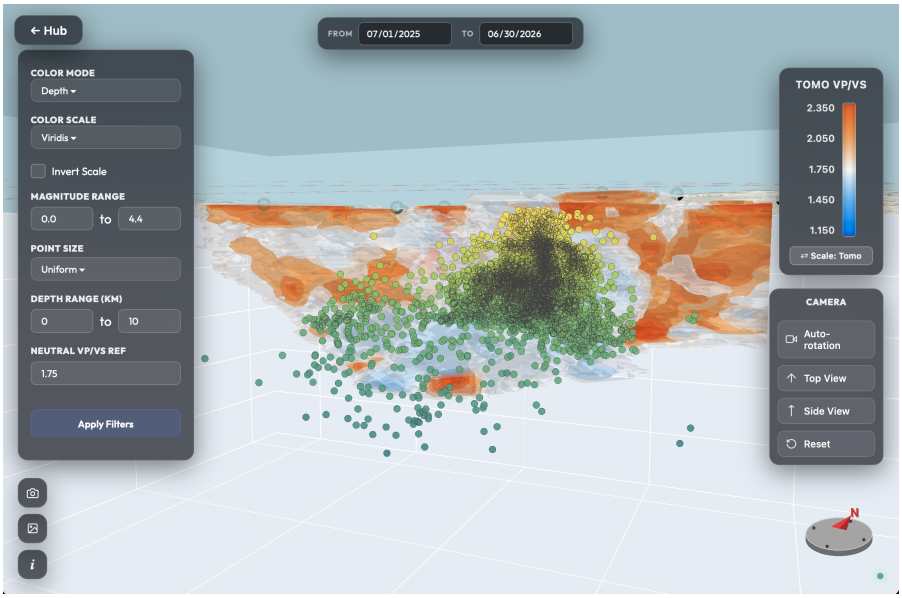
$$\text{ratio}_x = \frac{\Delta\lambda \cdot K \cdot \cos(\bar{\phi})}{\max(d_{\text{lon}}, d_{\text{lat}})}, \quad \text{ratio}_y = \frac{\Delta\phi \cdot K}{\max(d_{\text{lon}}, d_{\text{lat}})} \tag{5.1}$$

where $\Delta\lambda$ and $\Delta\phi$ are the longitude and latitude spans, $K = 111.32$ km/° is the equatorial conversion factor, and $\bar{\phi}$ is the mean latitude. This adjustment preserves accurate spatial proportions.

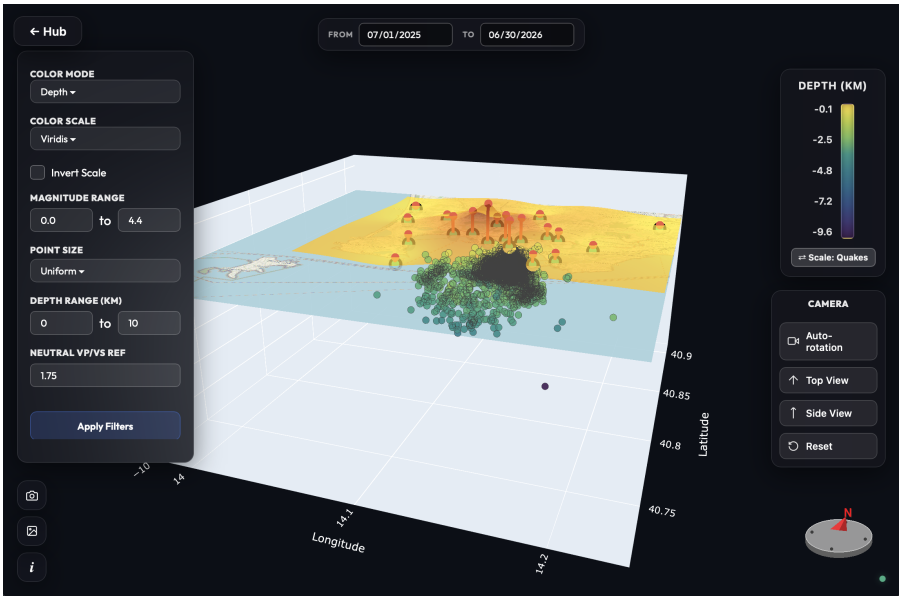
The 3D workspace (Figure 5.2) enables visualization of earthquake cluster geometries. Integrating a 3D tomography model (Figure 5.2b) correlates seismicity with geological structures, while vertical uplift columns (Figure 5.2c) allow direct comparison between surface deformation and underlying seismic activity.



(a) Point cloud below surface map.



(b) Added tomography layer.



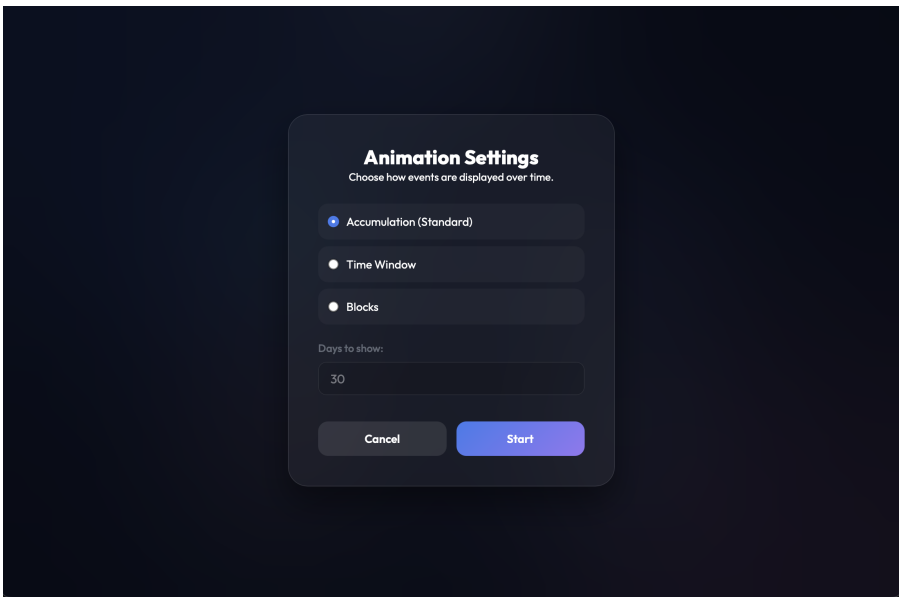
(c) Ground uplift columns.

Figure 5.2. The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c).

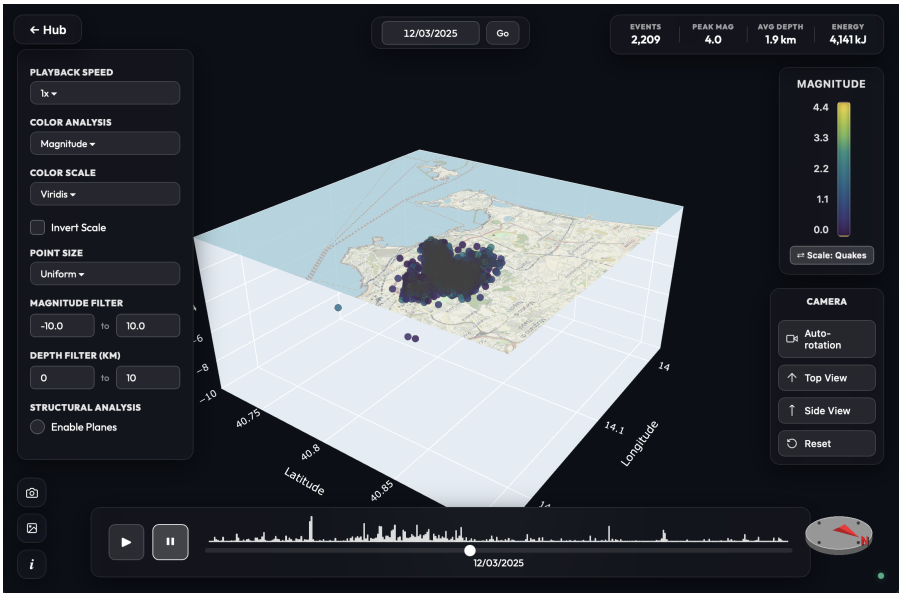
5.3 3D Animated View (4D Visualisation)

The animated view adds time as a fourth dimension to observe the spatio-temporal evolution of seismic sequences.

The 4D visualisation engine offers three temporal modes to analyse seismic evolution: **Accumulation** (showing all events up to the current frame), **Sliding window** (displaying events within a moving time frame), and **Block mode** (rendering discrete, non-overlapping time intervals). To ensure smooth playback and avoid memory overload, the system precomputes temporal indices and updates the 3D scene dynamically without reloading the entire figure.



(a) Temporal window configuration dialog.



(b) Active 4D playback view showing the time-sliced hypocentral point cloud.



(c) Dynamic cylinder grid partitioning with automated structural plane fitting during 4D evolution.

Figure 5.3. The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c).

Prior to starting the animation, the time parameters (such as step size and frame rate) are configured via a dedicated dialog window (Figure 5.3a). The resulting 4D view (Figure 5.3b) allows interactive exploration of the chronological sequence. Furthermore, the volume can be partitioned into a spatial grid of vertical cylinders (Figure 5.3c). This subdivision was implemented to better understand the spatial

distribution of the data, as partitioning the volume effectively isolates and subdivides the complex fault geometry into local, simpler planar segments. As earthquakes occur inside these cylinders, the system dynamically calculates and renders these local structural fault planes in real-time, allowing users to visualize how seismic events evolve over time across the identified planar structures.

5.4 3D Cluster Analysis View

The cluster analysis mode integrates spatial machine learning with structural geological modeling.

5.4.1 Cluster Visualisation & Interactive Tuning

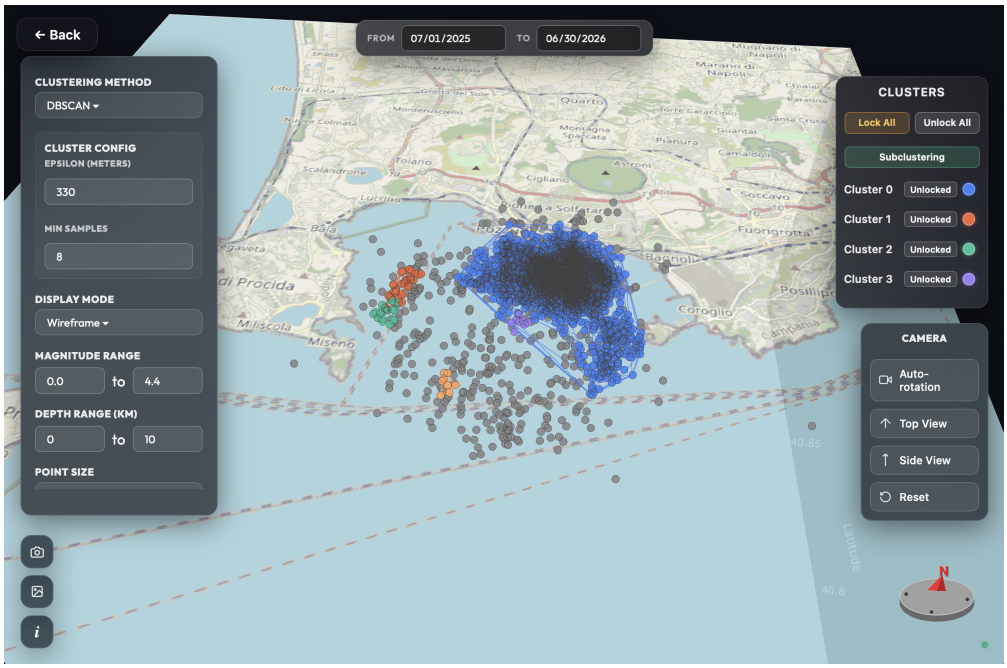
To provide a comprehensive geometrical understanding of seismogenic structures, every identified cluster is simultaneously visualized across four complementary representations within the 3D WebGL scene:

1. **Wireframe Convex Hulls (`Scatter3d`)**: Outer boundary envelopes highlighting the spatial footprint and volumetric extent of each distinct cluster.
2. **Triangulated Solid Meshes (`Mesh3d, alphahull=0`)**: Smooth, semi-transparent volumetric convex hulls enclosing the clusters.
3. **Structural Fault Planes (`Mesh3d`)**: Planar surfaces calculated via RANSAC robust estimation coupled with Singular Value Decomposition (SVD) refinement. Each plane displays orientation vectors alongside precise geological strike and dip angles upon mouse hover.
4. **Clustered Event Spheres**: Discrete event spheres colored by cluster label using categorical palettes, with unclustered background noise explicitly assigned neutral grey.

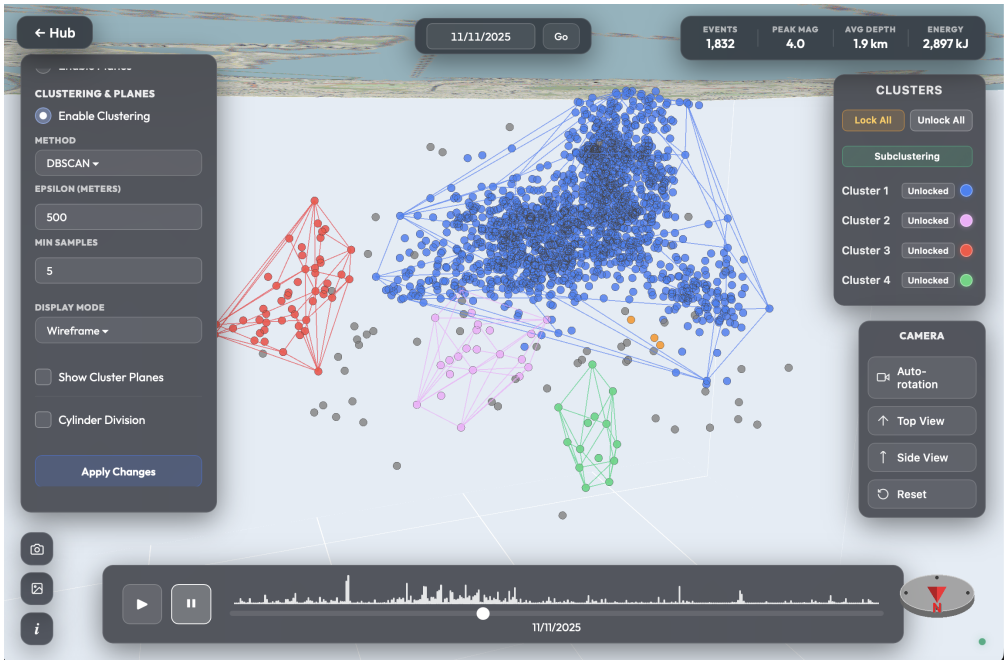
When users interactively adjust spatial clustering hyperparameters, such as neighborhood radius ε, minimum samples threshold `min_samples`, or cluster count K, via the side control panel, the frontend dispatches a `get_cluster_update` WebSocket message. The Python backend asynchronously recomputes cluster labels and structural planes without blocking the event loop, returning the newly generated Plotly trace definitions. The frontend then replaces the existing traces in the WebGL scene, thereby updating the visualization while preserving the user's current camera perspective.

5.4.2 Sub-Clustering

To analyze hierarchical structures, researchers can perform secondary sub-clustering on specific data subsets. By utilizing the **locked clusters** feature, users can preserve previously identified and validated groups; the algorithm will then selectively reanalyze and sub-divide all the remaining unlocked clusters simultaneously. The refined results can subsequently be applied back to merge cleanly into the parent view.



(a) Static 3D workspace with hulls and planes.

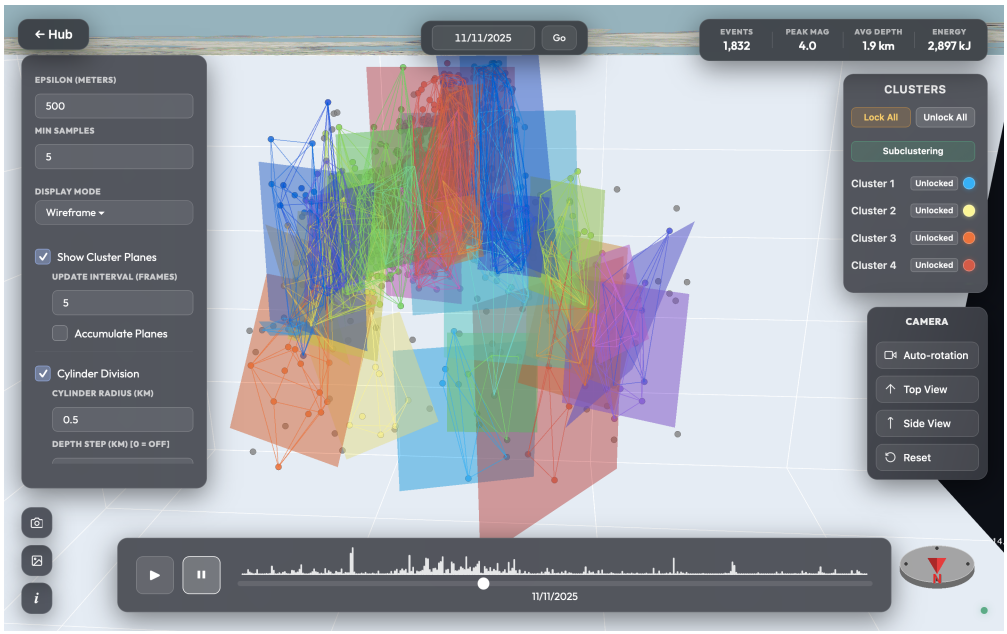


(b) Animated clustering with Global Label Matching.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1).



(c) Animated clustering with RANSAC+SVD planes.



(d) Unified 4D cylinder partitioning and plane fitting.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2).

Figure 5.4 provides a comprehensive overview of the clustering tools. Figure 5.4a shows the standard 3D clustering interface. Figure 5.4b displays how clustering evolves over time; to ensure visual consistency, a tracking algorithm guarantees that each cluster maintains the same color across all animation frames. Finally, Figure 5.4c and Figure 5.4d combine the animated clusters with dynamically calculated fault planes and spatial cylinders. This combination allows researchers to easily observe how geological structures evolve and rotate over time.

5.5 Additional Data Layers & View Caching

5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations

Seismic Viewer integrates three auxiliary data types processed via `core/layer_utils.py`:

- **Seismic Tomography (V_p/V_s Ratio)**: The raw tomographic data, originally provided in UTM Zone 33N coordinates, is converted to geographic coordinates and mapped onto a dense $60 \times 50 \times 40$ 3D grid. Because measurements only exist at specific surveyed points, the software must estimate the values for the surrounding empty space. It achieves this using *Inverse Distance Weighting*, a spatial interpolation technique. The value of any unknown point is calculated as a weighted average of its $k = 16$ nearest known neighbors. The closer a neighbor is, the stronger its influence on the final calculation, scaling inversely with the square of the distance ($p = 2$):

$$V(\mathbf{x}) = \frac{\sum_{i=1}^{k} w_i \cdot V_i}{\sum_{i=1}^{k} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)^2} \tag{5.2}$$

  However, Inverse Distance Weighting can produce unrealistic mathematical artifacts in empty regions located very far from actual measurement stations. To solve this, the system applies a *distance-decay* function, which acts as a spatial fade-out effect. As the distance from the nearest station increases, this function smoothly blends the estimated value back into the standard natural background value of the Earth's crust (1.75):

$$V_{\text{final}}(\mathbf{x}) = \alpha(\mathbf{x}) \cdot V_{\text{IDW}}(\mathbf{x}) + (1 - \alpha(\mathbf{x})) \cdot 1.75 \tag{5.3}$$

  where $\alpha(\mathbf{x}) = \exp(-(d_{\min}/\sigma)^2)$ with $\sigma = 0.15$ acts as the blending weight, decreasing smoothly from 1 (near known data) to 0 (far away).

- **GNSS Ground Deformation (Uplift)**: To visualize ground movement, the system reads historical displacement records and matches them with the known locations of the monitoring stations. This provides the exact vertical movement at any given time. Because data is only available at these specific points, the software estimates the ground movement in the empty areas between them using the same distance-based averaging (IDW) described above. The final result is displayed on the map either as a colored 2D grid or as 3D columns rising from the surface.

- **Monitoring Stations**: Permanent station positions are plotted as labelled markers, aiding spatial evaluation of catalogue completeness thresholds.

5.5.2 View Caching and Navigation

To ensure faster navigation between complex workspaces, the backend bridge maintains an in-memory view cache (`_html_store`, max 15 entries) keyed by a composite tuple of file path, view style, animation/windowing parameters, clustering configuration, and locked cluster states. A navigation history stack (`_history_stack`) allows instantaneous back and forward navigation in the clustering view while preserving exact parameter context.

Chapter 6

Clustering and Structural Analysis Algorithms

This chapter formalises the algorithms used in Seismic Viewer's analytical engine, presenting their mathematical formulations and optimization strategies.

6.1 Coordinate Projection & Spatial Clustering Methods

6.1.1 Metric Space Projection

To accurately calculate distances and geometric shapes, the system cannot use raw geographic coordinates (longitude λ and latitude ϕ) because they are measured in spherical degrees. Furthermore, the physical distance covered by one degree of longitude shrinks as one moves away from the equator. To solve this, before performing any clustering, the engine converts the spherical coordinates into a flat, 3D Cartesian grid measured in kilometers $(x, y, z)_{\text{km}}$. It does this by creating a localized flat map centered exactly on the average location of the dataset (λ_0, ϕ_0). This conversion, known as an equirectangular approximation, is calculated as follows:

$$x = (\lambda - \lambda_0) \cdot K \cos(\phi_0), \quad y = (\phi - \phi_0) \cdot K, \quad z = z_{\text{depth}} \tag{6.1}$$

where $K = 111.32$ km/° represents the average length of one degree of latitude, and z_{depth} represents the depth of each individual earthquake (measured as positive downwards).

6.1.2 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning

Seismic Viewer implements three complementary clustering paradigms:

1. **DBSCAN**: Scikit-learn's density-based spatial clustering (`DBSCAN`) isolates high-density seismogenic structures according to neighbourhood radius $\varepsilon_{\text{km}} = \varepsilon_{\text{m}}/1000$ and `MinSamples`. Points lacking sufficient density reachability are marked as noise (label -1).

2. **MiniBatchKMeans**: For centroid partitioning, `MiniBatchKMeans` optimizes K clusters across random data subsets ($b = 1024$), reducing iteration complexity from $O(NK)$ to $O(bK)$. To prevent outlier inclusion, a post-clustering distance filter reclassifies points exceeding $d_{\max}$ from their centroid as noise:

```
if method == 'kmeans' and max_dist > 0:
    max_d = max_dist / 1000.0
    for i in range(n_clusters):
        idx = (labels == i)
        dists = np.sqrt(np.sum((pts_km[idx] - centroids[i])
    **2, axis=1))
        display_labels[np.where(idx)[0][dists > max_d]] = -1

```

Listing 6.1. Post-clustering distance filter for KMeans.

3. **Cylinder Partitioning**: To ensure stable cluster assignments during 4D temporal animations without color swapping, the study volume is partitioned into a regular 2D grid of fixed vertical cylinders (60×50 cells). Events are assigned to the nearest cylinder centroid via KDTree query. Cylinders accumulating more than 5 events constitute stable, depth-resolved clusters across frames.

Table 6.1. Comparison of spatial clustering approaches in Seismic Viewer.

Property	**DBSCAN**	**MiniBatchKMeans**	**Cylinder Partition**
Cluster count / shape	Automatic / Arbitrary	User-defined (K) / Spherical	Grid-dependent / Cylindrical
Noise handling	Native (label -1)	Distance threshold ($d_{\max}$)	Density threshold (≥ 5 events)
Primary use case	Unknown structures, swarms	Fixed centroid tracking	4D animation, fault plane tracking

6.2 Automatic Parameter Tuning

Within the Seismic-Viewer application, these auto-tuning algorithms are specifically triggered during the interactive *Subcluster Analysis* phase. Instead of forcing the user to manually guess the optimal clustering parameters for isolated subsets of events, the engine dynamically calculates them for all currently *unlocked* targets by analyzing their local spatial geometry:

- **DBSCAN Auto-ε**: The system calculates the distance from each earthquake to its nearest neighbors. When these distances are sorted from smallest to largest and plotted on a graph, they form a curve. The ideal search radius ε is located exactly at the point of maximum curvature, where the graph bends the most. To find this optimal point mathematically, the software draws a straight imaginary line from the start to the end of the curve, and then finds

the specific point on the curve that is furthest away from that straight line. This maximum perpendicular distance is calculated as follows:

$$i^* = \arg\max_i \left\| \mathbf{p}_i - \left(\frac{\mathbf{p}_i \cdot \hat{\mathbf{v}}}{\|\hat{\mathbf{v}}\|^2} \right) \hat{\mathbf{v}} \right\| \tag{6.2}$$

where $\mathbf{p}_i$ represents a single point on the curve, and $\hat{\mathbf{v}}$ represents the straight line connecting the curve's endpoints.

- **MiniBatchKMeans Auto-K**: To find the optimal number of clusters K, the system must balance grouping accuracy against the complexity of having too many clusters. It runs the algorithm multiple times, testing every value from 2 up to 10 clusters. For each test, it calculates the *inertia*, a mathematical score measuring how tightly packed the resulting clusters are. When these inertia scores are plotted on a graph, they form a downward-sloping curve, because increasing the number of clusters naturally forces them to be smaller and tighter. The system's goal is to find the point of maximum curvature of this curve, which represents the point of diminishing returns where adding more clusters no longer provides a meaningful improvement. To pinpoint this exact value automatically, the software applies the identical perpendicular distance formula (Equation 6.2) to the inertia curve: it draws an imaginary straight line connecting the first test ($K = 2$) to the last test ($K = 10$), and selects the cluster count that sits furthest from this straight line.

6.3 Structural Plane Fitting & Geometric Synthesis

6.3.1 RANSAC Robust Plane Estimation

To identify structural fault surfaces within noisy hypocentral clouds, the RANSAC algorithm iteratively samples point triplets $\{P_1, P_2, P_3\}$, computing candidate normals:

$$\vec{n}_{\text{cand}} = \frac{(P_2 - P_1) \times (P_3 - P_1)}{\|(P_2 - P_1) \times (P_3 - P_1)\|} \tag{6.3}$$

Inliers within a specific distance tolerance are tallied across 100 iterations. This tolerance is context-dependent: it is set to a fixed 0.15 km for isolated subclusters, whereas it scales dynamically as $\max(0.02, 0.1 \cdot r_{\text{cyl}})$ km when operating on continuous cylinder grids (where r_{cyl} is the search cylinder's radius). The candidate maximizing the inlier count and minimizing the RMS residual is then selected.

6.3.2 SVD Refinement & Strike/Dip Mechanics

While RANSAC successfully identifies the best subset of points belonging to a fault, its initial plane is generated using only three random points. To compute the absolute best-fitting plane that averages across *all* the identified inliers, the engine applies Singular Value Decomposition (SVD).

First, the software calculates the geometric center (centroid) of the inliers and shifts all points so they are centered around zero, forming a matrix A:

$$A = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} & z_1 - \bar{z} \\ \vdots & \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} & z_n - \bar{z} \end{bmatrix} = U\Sigma V^T \tag{6.4}$$

SVD analyzes this point cloud to find its three primary directions of spread. The direction where the cloud is thinnest (the minimum variance, corresponding to the smallest singular value σ_3) is chosen as the normal vector $\vec{n}$ of the fault plane.

Additionally, SVD provides singular values $(\sigma_1, \sigma_2, \sigma_3)$ that measure the general shape of the cloud. The software uses these to calculate a *Planarity* score:

$$P = \frac{\sigma_2 - \sigma_3}{\sigma_1} \tag{6.5}$$

A high planarity score indicates a clean, flat fault surface, while a low score suggests a scattered or disorganized distribution.

Finally, to translate this mathematical plane into standard geological metrics, the normal vector $\vec{n} = [n_x, n_y, n_z]$ is forced to point downwards into the earth ($n_z \geq 0$). Standard trigonometry is then applied to extract the fault's orientation:

$$\text{Dip}: \quad \delta = \arccos(n_z) \cdot \frac{180^\circ}{\pi} \tag{6.6}$$

$$\text{Dip Direction}: \quad \theta_{\text{DD}} = \text{atan2}(n_x, n_y) \bmod 360^\circ \tag{6.7}$$

$$\text{Strike}: \quad \phi_s = (\theta_{\text{DD}} - 90^\circ) \bmod 360^\circ \tag{6.8}$$

The *Dip* measures the tilt angle relative to the surface, the *Dip Direction* is the geographic direction of the tilt, and the *Strike* is the geographic orientation of the horizontal fault line itself (calculated using the right-hand rule). Planes render as 20×15 triangulated grids constructed along orthogonal basis vectors $\vec{u}, \vec{v}$:

$$\vec{u} = \frac{\vec{n} \times \hat{z}}{\|\vec{n} \times \hat{z}\|}, \quad \vec{v} = \frac{\vec{n} \times \vec{u}}{\|\vec{n} \times \vec{u}\|} \tag{6.9}$$

6.4 Temporal Stability & Parallel Execution

During 4D animated clustering, independent per-frame executions can cause color flickering due to non-deterministic cluster label indexing. The `match_labels_globally()` routine enforces temporal stability by tracking a global label map across consecutive frames: each cluster inherits the majority global label of its constituent events (sampled stochastically using a maximum threshold of 200 points for large clusters to maintain $O(1)$ scaling with more or less 99% matching accuracy).

To process multi-frame RANSAC+SVD plane evaluations efficiently, the engine distributes calculations across CPU cores via `ProcessPoolExecutor` (capped at $\min(\texttt{cpu_count}(), 8)$). Worker processes utilize the shared memory initializer pattern to store `pts_km` once at pool creation, eliminating inter-process serialization bottlenecks across hundreds of temporal frames.

Chapter 7

Development Process and Challenges

This chapter documents the software engineering process behind Seismic Viewer, detailing the development timeline, the iterative implementation methodology, major technical challenges and key architectural decisions.

7.1 Development Timeline & Iterative Methodology

The project evolved through continuous iterations. As depicted in Figure 7.1, development progressed through six distinct phases: (1) 3D visualization prototyping, (2) pywebview bridge integration and clustering, (3) UI/UX redesign and INGV API integration, (4) binary streaming and cylinder grid implementation, (5) integration of tomography and GNSS station layers, and (6) modular package refactoring and testing.

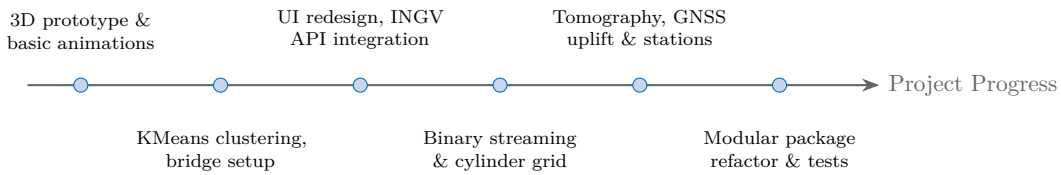


Figure 7.1. Development milestones highlighting the iterative implementation methodology.

The software was built using an iterative design process, where features were progressively prototyped and optimized. This was essential because visualizing seismological data and analyzing clusters in real-time required continuous testing, feedback, and adjustments. As the application grew in complexity, its underlying codebase had to be completely reorganized twice: first, by splitting a single massive script into a clean, modular package, and later by upgrading to a FastAPI and WebSocket system to stream data significantly faster.

7.2 Key Technical Challenges and Solutions

7.2.1 Rendering & UX Optimization: Camera Persistence, Binary Streaming, and Transitions

Camera State Preservation. Initial prototypes suffered from view resets during dynamic filtering because Plotly.js re-initialized scene parameters on every `react()` call. Adding `uirevision='constant'` to all 3D layouts permanently resolved this, preserving user camera orientation, zoom, and translation across dynamic reclustering:

```
fig.update_layout(
    uirevision='constant',
    scene=dict(
        dragmode='turntable',
        camera=dict(eye=dict(x=1.25, y=1.25, z=1.25), center=dict(x=0, y=0, z=0))
    )
)
```

Listing 7.1. Camera state persistence via `uirevision`.

JSON Bottleneck & Binary Streaming. Early prototypes serialized large seismic swarms via JSON strings, causing massive garbage collection freezes in the browser engine during parsing. This bottleneck was entirely resolved by migrating the communication protocol to a raw Float32 binary buffer. As detailed previously in Section 3.4.3, decoding this structure directly via JavaScript's `Float32Array` drastically optimized both payload size and parsing speed, enabling fluid rendering.

White-Flash Mitigation. Navigating between pywebview HTML views originally produced jarring white flashes before CSS loading. A dark overlay injection script (`_inject_dark_overlay()`) now inserts an opaque full-screen container prior to navigation, which is smoothly faded out.

7.2.2 Algorithmic Resilience & Memory Management: Temporal Flickering, API Isolation, and Caching

Chromatic Flickering in 4D Clustering. Because unsupervised algorithms assign arbitrary cluster indices across independent frames, colors swapped between steps. Implementing the Global Label Matching routine (Section 6.4) stabilized tracking via majority voting across consecutive frames.

INGV API Resilience. The GOSSIP service occasionally encounters timeouts or missing annual packets. The online importer isolates requests into per-year `try...except` loops, logging warnings for incomplete years while processing all available records without halting the pipeline.

Session Memory Bounding. To prevent memory inflation during multi-hour exploration sessions, the backend enforces FIFO eviction across `_data_store` (max 30 sessions) and `_html_store` (max 15 views). Combined with `strip_data()` point cloud separation, RAM utilization remains bounded below 500 MB under heavy multi-view concurrency.

7.3 Code Quality and Architecture

The final software is organized into a clean, layered structure. The backend is written in statically typed Python and divided into 12 distinct modules, while the frontend is built with JavaScript, HTML, and CSS. The codebase is designed so that data flows in a single direction: starting from the main application script (`app.py`), passing through the core logic (`engine.py` and `bridge.py`), and ending in specialized utility files (`loader.py` and `geo_utils.py`). This separation makes the code easy to maintain and ensures that different parts of the application remain fully independent. Finally, the system uses multiple caching strategies (on disk with `pickle`, in memory with `lru_cache`, and for active sessions in `_html_store`). These caches are kept completely separate from one another to prevent bugs where old or incorrect data might accidentally be displayed to the user.

Chapter 8

Conclusions and Future Work

8.1 Summary of Contributions

Moving beyond the constraints of traditional seismological software (Section 2.3), this project bridges the gap between raw geophysical data and real-time visual analysis through eight primary contributions:

1. **Dual-Engine Architecture**: An asynchronous bridge integrating a `pywebview` native window with a high-throughput FastAPI local server (Chapter 3), enabling native file dialogs and desktop integration without the memory overhead of Electron.

2. **Binary Streaming Protocol**: A custom `Float32` binary buffer transport that reduces network overhead by 90% compared to JSON and drops frontend decode latencies.

3. **Multi-Resolution Topographic Rendering**: A vector-cached Mercator map engine (Section 4.4.1) that dynamically projects OpenStreetMap/Smopy raster tiles into 3D meshes (`Mesh3d`).

4. **Temporal Windowing & 4D Animation**: An interactive temporal playback system with dual data storage (`_data_store` for 3D/4D and `_html_store` for 2D) supporting temporal sliding and animated clustering (Figure 5.3).

5. **Real-Time Online Data Ingestion**: Direct integration with the official INGV GOSSIP web services API (Section 4.3), featuring automated annual chunk fetching, fault tolerance, and comprehensive magnitude/quality filtering.

6. **Auxiliary Geophysical Layering**: Native overlay support for 3D V_p/V_s seismic tomography cubes and 2D GNSS vertical displacement station grids, bridging raw seismicity with deep crustal deformation.

7. **Interactive & Animated Clustering**: Real-time spatial clustering (DBSCAN, `MiniBatchKMeans`, and 4D cylinder grid partitioning) executed locally in Python and pushed over WebSockets (`get_cluster_update`) to prevent camera reset during visual tuning (Figure 5.1).

8. **Outlier-Resilient Fault Plane Fitting**: Automated extraction of tectonic structures using RANSAC outlier rejection coupled with SVD orientation refinement (Section 6.3), directly computing dip, dip direction, and strike.

8.2 Validation & Performance Assessment

The system was evaluated using the dense INGV Campi Flegrei caldera catalog (∼30,000 events, 2005-2024), characterized by intense bradyseismic swarms and complex shallow fracturing. Quantitative profiling confirms that Seismic Viewer achieves consistent 60-120 FPS rendering performance on standard desktop hardware (M-series Apple Silicon and NVIDIA RTX GPUs) when visualizing datasets exceeding 50,000 simultaneous points.

Because the true position and orientation of underground faults cannot be directly observed, the plane fitting algorithms (RANSAC and SVD) were validated using synthetic datasets where the exact geometric structures were known. These datasets were generated by mathematically defining 3D planes of points at specific strike and dip angles, and subsequently introducing random Gaussian noise and scattered outliers to simulate realistic seismic measurement errors. These tests demonstrated that the algorithm correctly identifies fault orientations with high precision, consistently calculating the expected dip and strike angles even when the data was heavily contaminated by simulated noise. Furthermore, the global label matching algorithm (Section 6.4) successfully eliminated temporal color flickering across 4D animated cluster sequences while maintaining fast computation times.

8.3 Future Research & Architectural Extensions

While Seismic Viewer achieves interactive responsiveness for current regional catalogs, several areas present opportunities for future development:

- **Rendering & Web Deployment**: As dataset sizes continue to grow, migrating the frontend to specialized geospatial rendering engines (such as Deck.gl or a custom WebGL/Three.js pipeline) could allow for progressive level-of-detail rendering, further increasing the maximum point limits and enabling a fully browser-based web version.

- **Real-Time Monitoring Integration**: The architecture currently analyzes static or periodically updated catalogs. Integrating continuous data streaming protocols (like SeedLink or FDSN web services) could transition the system into a live surveillance dashboard for monitoring rooms.

- **Advanced Geophysical Layers**: Future iterations could expand the analytical capabilities by integrating focal mechanisms to visually represent fault in 3D, as well as Coulomb static stress transfer maps to analyze tectonic stress variations.

8.4 Concluding Remarks

In conclusion, Seismic Viewer demonstrates how modern web technologies can be effectively combined with scientific Python libraries to create powerful and accessible visualization tools. The development of this project highlights that careful software architecture can significantly improve the workflow of researchers analyzing large datasets. By providing a highly interactive interface for exploring seismological catalogs, this application offers a solid and practical foundation for investigating tectonic structures and monitoring ongoing geological unrest, such as the bradyseismic activity at the Campi Flegrei caldera.

Appendix A

Supplementary Technical Details

This appendix provides a formal specification of the data structures and the core algorithmic logic utilized within Seismic Viewer. Standardizing the input and internal formats ensures that the system can easily parse external catalogs and efficiently render them, while the computational pipeline ensures robust data analysis. By documenting these structures and algorithms, future extensions to the software can maintain compatibility with the existing core architecture.

A.1 Seismic Data Structures

The design of Seismic Viewer places a strong emphasis on maintaining a clear separation between the external data representation and the internal state used for high-performance computation. This section outlines both formats in detail, illustrating how raw textual data is efficiently translated into numerical matrices.

A.1.1 Input Catalog Format

Seismological data must be structured in a CSV following the specifications detailed in table A.1. The schema requires specific temporal and spatial fields to ensure correct localization within the 3D WebGL environment. To accommodate the vast amount of data often recorded during seismic swarms, the format is kept as lightweight as possible. External catalogs, such as those provided by the INGV, must be pre-processed or mapped to this standard structure before ingestion. The mandatory fields include unique identifiers, precise origin timestamps, and geographical coordinates expressed in decimal degrees.

A.1.2 Internal Application State

Once parsed by the Python backend, the catalog is transformed into an internal state object optimized for rendering and clustering algorithms. The data is held in memory as multi-dimensional `numpy` arrays to facilitate the matrix operations required by DBSCAN and RANSAC. Transitioning from a row-based CSV structure to a column-based array format significantly improves memory locality and cache performance during heavy numerical computations. This design choice is fundamental to achieve the interactive frame rates needed in the application's GUI.

Table A.1. Expected schema for input seismic catalogs.

Field Name	Data Type	Description
`id`	`string`	Unique identifier for the seismic event
`time`	`ISO 8601 string`	Origin time of the event (e.g., 2024-01-01T12:00:00)
`latitude`	`float`	Event epicenter latitude in WGS84 decimal degrees
`longitude`	`float`	Event epicenter longitude in WGS84 decimal degrees
`depth`	`float`	Hypocentral depth in kilometers
`mag`	`float`	Magnitude of the event

Table A.2. Internal data representation in the processing pipeline.

Variable	Type	Purpose
`coordinates`	`numpy.ndarray`	An $N \times 3$ matrix storing projected UTM coordinates and depth
`magnitudes`	`numpy.ndarray`	An N-length array used for scaling the visual markers
`timestamps`	`numpy.ndarray`	Unix epoch timestamps used for temporal filtering and animation
`labels`	`numpy.ndarray`	Cluster assignments (-1 for noise, $0 \dots K$ for structural groups)

A.2 Algorithmic Pipeline

The objective of the spatial analysis module is to extract dominant structural orientations from unorganized point clouds representing earthquake hypocenters. This is achieved through a multi-stage pipeline that first isolates dense clusters of seismicity and then determines the geometric properties of the underlying fault structures.

A.2.1 Spatial Clustering

The system employs DBSCAN to group seismic events that occur in close spatial proximity, effectively separating them from isolated, background seismicity. The clustering logic evaluates distances in three dimensions using a Euclidean metric, allowing it to identify arbitrarily shaped clusters without requiring the user to specify the number of groups beforehand. The core implementation relies on the highly optimized `scikit-learn` library to process tens of thousands of points efficiently.

A.2.2 Fault Plane Extraction

For each identified spatial cluster, the system estimates the best-fitting geometric plane. This process relies on the RANSAC algorithm to iteratively sample subsets of the cluster and discard outliers that do not conform to a planar structure. Once the set of robust inliers is identified, SVD is applied to the centered coordinates. The normal vector of the fault plane corresponds to the right singular vector associated with the smallest singular value, providing a mathematically rigorous definition of the structure's orientation in 3D space.

Bibliography

[1] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996.

[2] Aurélien Géron. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Inc., 3rd edition, 2022. ISBN 978-1098125974.

[3] G. Giordano, G. Alfonsi, F. Salvini, F. Bianco, C. Chiarabba, M. Di Vito, and A. Tramelli. Birth and growth of a volcanotectonic fault during the current volcanic unrest at Campi Flegrei caldera (Italy). *Communications Earth & Environment*, 6(1), 2025. doi: 10.1038/s43247-025-02803-2.

[4] Istituto Nazionale di Geofisica e Vulcanologia. Osservatorio Vesuviano – Portale GOSSIP. https://terremoti.ov.ingv.it/gossip/, 2024. Accessed: 2026-07-15.

[5] Ville Satopaa, Jeannie Albrecht, David Lo, and Matthew A. Ward. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *31st International Conference on Distributed Computing Systems Workshops*, pages 166–171. IEEE, 2011. doi: 10.1109/ICDCSW.2011.20.